Replicating and Extending
How Powerful are Graph Neural Networks?

Sobhan Salarian
Macquarie University
COMP8240: Applications of Data Science
shaygan.sobhan79@gmail.com

Abstract

We replicate the graph classification results of Xu et al. (2019) on IMDB-BINARY and IMDB-MULTI using the authors' code, and test the Graph Isomorphism Network (GIN) on new data. Replicated accuracy falls within one reported standard deviation of the paper on both datasets. Our first new dataset, built from The Movie Database (TMDb), produced a large drop in accuracy, which we traced to a construction flaw: every graph was a clique, leaving node count as the only signal. We rebuilt the data as actor ego-networks and also ran the existing ENZYMES benchmark. On ego-networks GIN and GraphSAGE tied at well above the node-count baseline (74.4% and 59.7% for GIN), while on ENZYMES GIN clearly won, 48.5% to 33.3%. An LLM judge agreed strongly with the genre labels ($\kappa \geq 0.946$) and its disagreements exposed a date filter bug in our scraper.

1 Description of Paper and Justification

Xu et al. (2019)[1] ask how expressive message-passing Graph Neural Networks (GNNs) are, meaning which graphs they can tell apart. They prove that the Weisfeiler-Lehman (WL) test (Weisfeiler and Lehman, 1968) is an upper bound for this class of model, and that mean and max aggregation, as used in GCN (Kipf and Welling, 2017) and GraphSAGE (Hamilton et al., 2017), are strictly weaker. They then propose GIN, which sums neighbour features and passes them through a multi-layer perceptron (MLP), and show that it reaches the WL bound.

The theory in brief. The WL test gives every node a label and then repeats one step: each node's new label is a summary of its old label together with the multiset of its neighbours' labels. A multiset is a set that keeps repeats, so two neighbours with the same label count twice. If two graphs end up with different collections of labels, they are not isomorphic. A message-passing GNN does the same thing with feature vectors, so it can never separate two graphs that WL cannot. It matches WL if its aggregation, update and graph readout never map two different multisets to the same output. Sum aggregation followed by an MLP can meet this condition. Mean keeps only the proportions of each feature, so a neighbourhood $\{a, b\}$ looks the same as $\{a, a, b, b\}$. Max keeps only which features occur, so $\{a, b\}$ looks the same as $\{a, a, a, b\}$. GIN updates node v at layer k as

$$h_v^{(k)} = \mathrm{MLP}^{(k)}\big((1+\epsilon^{(k)})\, h_v^{(k-1)} + \textstyle\sum_{u \in \mathcal{N}(v)} h_u^{(k-1)}\big),$$

and builds the graph's representation by summing node features at every layer and combining the layers. The paper also reports that GIN fits the training data almost perfectly, while the weaker variants fit it less well. This gives a testable prediction: GIN should beat mean or max aggregation most clearly where repeated neighbour features matter.

Evaluation. The paper reports graph classification accuracy on nine benchmarks using 10-fold cross-validation. For each epoch, test accuracy is averaged across the folds; the single epoch with the highest average is selected, and the mean and standard deviation across folds at that epoch are reported. Every model has 5 GNN layers and 2-layer MLPs and is trained with Adam at a learning rate of 0.01, halved every 50 epochs, for 350 epochs. The paper tunes hidden units, batch size and dropout per dataset; the official code's defaults are 64 hidden units, batch size 32 and dropout 0.5. GIN-0, the variant with ϵ fixed at 0, reaches 75.1 $\pm$ 5.1% on IMDB-BINARY and 52.3 $\pm$ 2.8% on IMDB-MULTI. The GraphSAGE approximation, Max–1-layer, reaches 72.3 $\pm$ 5.3% and 50.9 $\pm$ 2.2%.

Justification. ICLR is ranked A* by CORE,

[1] https://arxiv.org/abs/1810.00826

```
1000            <- number of graphs
20 0            <- 20 nodes, label 0
0 5 2 4 5 9 10  <- tag, degree, neighbours
```

Figure 1: Start of the IMDB-BINARY text file, with our annotations added.

and the paper is widely cited in graph learning. Its official code is public under the MIT licence, and its theory gives an explicit reason to expect a result to hold or fail, which suits an investigation on new data. The task also transfers well: any dataset in the same text format can be run with no change to the model.

2 Description of the Original Dataset

We replicate on two of the paper's nine benchmarks, because they are the two our new data mirrors. Both were released by Yanardag and Vishwanathan (2015) and ship with the official repository.[2] In each graph, nodes are actors and an edge means two actors appeared in a film together. Each graph is one actor's ego-network, labelled by genre: Action or Romance for IMDB-BINARY, and Comedy, Romance or Science Fiction for IMDB-MULTI. There are no node features, so node degree is used as the input feature (`--degree_as_tag`), which forces the model to rely on structure. Table 1 gives the statistics we computed from the files.

Each dataset is a single text file, shown in Figure 1. The first line is the graph count. Each graph then starts with its node count and label, followed by one line per node giving its tag, its neighbour count and its neighbours.

3 Replication of Original Work

Environment. The code targets PyTorch 0.4.1 and 1.0, but it ran on Python 3.11 with PyTorch 2.x with only deprecation warnings. A laptop CPU needed about 3.5 hours per dataset, so we first used a Google Colab T4 GPU. When Colab's quota ran out, we moved to a university server with an RTX 6000 Ada GPU. All scripts are in our public repository.[3]

Process. The official `main.py` trains one fold, and its output file is overwritten each epoch, so it cannot hold a learning curve. We wrote a driver, `run_gin.py`, that runs all 10 folds, parses each epoch's accuracy, and applies the paper's model-selection rule. We used the code's default hyperparameters for every dataset and did no tuning, because tuning per dataset would make the new-data results depend on our search rather than on the method. As in the paper, GraphSAGE is approximated in the same code with `--neighbor_pooling_type max --num_mlp_layers 1`. It is not the implementation of Hamilton et al. (2017).

Changes to the original code. We made one change. In the max-pooling path, `__preprocess_neighbors_maxpool` left an index tensor on the CPU, and on GPU the first GraphSAGE fold produced no output for over 12 minutes. Adding `.to(self.device)` resolved this and does not change the computation. Separately, `scikit-learn` is missing from the repository's requirements file, so our setup script installs it.

Results. Table 2 compares our results with the paper. All are within one of the paper's standard deviations. The ranking also holds on IMDB-BINARY, where GIN beats GraphSAGE by 1.3 points against the paper's 2.8. On IMDB-MULTI the two differ by 0.3 points in GraphSAGE's favour, where the paper has GIN ahead by 1.4. Both gaps are well inside one standard deviation, so we treat this as a tie and do not read it as a contradiction.

A separate GIN run on Colab gave $74.4 \pm 4.1\%$ and $50.5 \pm 2.3\%$, within about 0.5 points of the server. Only summary figures were kept from that run, so the table uses the server results, whose per-epoch curves are in the repository. Small differences from the paper are expected, because seeds and hardware differ. The selected epoch also varied between runs on IMDB-BINARY (13 on the server, 80 on Colab), since its averaged curve is very flat.

4 Construction of New Data

4.1 Existing Dataset: ENZYMES

ENZYMES (Borgwardt et al., 2005) contains 600 protein tertiary structures, 100 for each of 6 enzyme classes, and is not among the paper's nine benchmarks. Nodes are secondary structure elements with one of 3 labels, which we use as the input features. We downloaded it from the TUDataset collection (Morris et al., 2020).[4] Our proposal said this collection already matches the code's input format. That was wrong: it uses separate files for

[2] https://github.com/weihua916/powerful-gnns

[3] https://github.com/SobhanSalarian/Gin-Graph-application-of-data-science-

[4] https://chrsmrrs.github.io/datasets/

Dataset	Graphs	Classes	Avg. nodes	Density	Cliques	Degree seqs	Node-count acc.
IMDB-BINARY	1000	2	19.8	0.521	13.9%	537	58.1%
First scrape, film graphs, binary	996	2	19.2	1.000	100%	17	56.0%
Final ego-networks, binary	1000	2	45.1	0.508	0.0%	336	57.7%
IMDB-MULTI	1500	3	13.0	0.772	52.6%	387	40.1%
First scrape, film graphs, multi	1492	3	18.9	1.000	100%	18	40.5%
Final ego-networks, multi	1500	3	43.8	0.515	0.1%	633	46.3%
ENZYMES	600	6	32.6	0.160	0.5%	586	23.5%

Table 1: Dataset statistics computed from the files. Degree seqs is the number of distinct degree sequences. Node-count acc. is the 10-fold accuracy of a classifier that sees only the number of nodes.

	IMDB-BINARY	IMDB-MULTI
Paper GIN-0	75.1 ± 5.1	52.3 ± 2.8
Ours GIN-0	$\mathbf{73.9 \pm 2.7}$	$\mathbf{50.6 \pm 2.7}$
Paper Max–1-layer	72.3 ± 5.3	50.9 ± 2.2
Ours Max–1-layer	$\mathbf{72.6 \pm 3.0}$	$\mathbf{50.9 \pm 2.8}$

Table 2: Replication on the original data: test accuracy (%), mean $\pm$ standard deviation across 10 folds.

edges, graph membership and labels, so we wrote `convert_tu_dortmund.py` to produce the format in Figure 1.

4.2 Constructed Dataset: TMDb

Collection. We used the TMDb API (TMDb, 2026) to collect films released from 2016 onward, so none overlap with the 2015 originals. For each target genre we took up to 500 films by popularity with at least 20 votes, then fetched their credits. Films tagged with more than one target genre were dropped to keep labels unambiguous. We kept the top 20 billed actors and dropped films with fewer than 5. No human annotation was needed, because labels are TMDb's genre metadata. Section 5.1 checks their quality. Our first scrape did not apply the date limit: it sent `primary_release_date_gte` where the API expects `primary_release_date.gte`, and the API ignored the unknown parameter without an error, so films such as *Casablanca* were included. We fixed the parameter, re-scraped, and made the scraper stop if any film falls before the cut-off. The final data has 993 binary and 1,484 multi films, dated 14 January 2016 to 29 October 2026.

First construction, and its flaw. In the first scrape we built one graph per film, using its cast as nodes and linking actors who co-appeared in any sampled film of that genre. Before training we did not check the resulting structure closely, and the proposal's expectation of ego-network structure turned out to be wrong. Every pair of cast members has already co-appeared in that film, so the rule adds nothing, and all 2,488 graphs were cliques (Table 1). Two cliques of the same size are isomorphic, so node count was the only usable information.

Rebuilt construction. For the final data we followed Yanardag and Vishwanathan (2015) more closely. For each genre we form one collaboration graph, then take each actor's ego-network within it: the actor, their co-stars, and all links among those co-stars. Actors with only one film in a genre are excluded, because their ego-network is that film's clique. We sampled 500 graphs per genre at random from the 627 to 1,323 available.

The rebuilt data is close to the originals in density and has real structural variety (Table 1). There are two main differences. Graphs are more than twice the size of IMDB-BINARY's, because each film contributes up to 20 actors. Cliques are almost absent because of the exclusion above, against 13.9% and 52.6% in the originals. As in the originals, ego-networks overlap, so train and test graphs can share actors. An actor who appears in both genres has an ego-network in each, built from different films and given different labels.

Pipeline and checks. The data is built in three steps, each a script in our repository: `build_tmdb_dataset.py` collects films and credits, `build_ego_networks.py` forms the graphs from the saved metadata without further API calls, and `check_dataset.py` tests the result. The check reports density, the share of cliques, the number of distinct degree sequences, and the 10-fold accuracy of a classifier that predicts the most common class for each node count; these are the columns of Table 1. A dataset where most graphs are cliques, or where node count alone is nearly as accurate as a trained model, cannot test a structure-sensitive model, so every dataset now

Dataset	GIN-0	Max–1-layer	Nodes
First scrape, binary	55.9 ± 2.3	56.2 ± 2.3	56.0
First scrape, multi	41.0 ± 2.3	41.0 ± 2.2	40.5
Ego, binary	74.4 ± 2.5	74.0 ± 1.9	57.7
Ego, multi	59.7 ± 4.4	57.6 ± 5.0	46.3
ENZYMES	48.5 ± 7.0	33.3 ± 5.6	23.5

Table 3: Results on new data: test accuracy (%). Nodes is the node-count baseline: 10-fold accuracy using only the number of nodes.

passes this check before any training.

5 Results on New Data

Hyperparameters and protocol are unchanged, and the code is the same as in the replication. Table 3 gives the results. The node-count column is the reference point: accuracy above it requires information from graph structure.

First-scrape film graphs. GIN fell by 18.0 points on the binary task and 9.6 on the multi task compared with our replication, and tied with GraphSAGE. Both models sit at the node-count baseline, so they learned cast size and nothing else. With no structure to separate graphs, this is what the theory predicts, and it tests nothing about GIN. It does show the value of running a structure-free baseline before training.

Ego-networks. Unlike the first scrape, both models now sit well above the node-count baseline: GIN by 16.7 points on the binary task and 13.4 on the multi task, so they are using structure. Binary accuracy is close to our replication (74.4% against 73.9%). Multi accuracy is 9.1 points higher than the replication, but this does not show GIN doing better, because these graphs are larger and the baseline itself is 6.2 points higher. GIN leads GraphSAGE by 0.4 and 2.1 points, both within one standard deviation, so we treat the models as tied on this data.

ENZYMES. This is the only dataset with a clear gap. GIN reached 48.5% and GraphSAGE 33.3%, a difference of 15.2 points, more than twice either standard deviation. Both beat the node-count baseline (23.5%) and chance (16.7%). Mean training accuracy at the selected epoch was 99.7% for GIN and 39.2% for GraphSAGE, so GraphSAGE underfits while GIN overfits heavily. This fits the paper's theory: with only 3 node labels, neighbours often share a label, and max aggregation cannot count them. We did not test this cause directly, and the 1-layer MLP may also contribute.

Limits. The selection rule picks the epoch using test folds, as in the paper, so all figures are somewhat optimistic. Train and test ego-networks can also share actors.

Hardware. All runs used the GPU except GraphSAGE on the ego-networks. Max pooling was slow on these dense graphs: about 26 seconds per epoch on the shared GPU, against 13 on one CPU core. We ran those two jobs on CPU, 10 folds at a time with one thread each, which took under 2 hours per dataset rather than an estimated 25. Seeds and settings were unchanged, but their results are not numerically identical to what a GPU run would give.

Learning curves. Figure 2 shows test accuracy averaged over the folds at every epoch. On the original IMDB-BINARY, both curves peak within the first 15 epochs; GIN then stays near 72% while GraphSAGE drifts down to about 69%. On the binary ego-networks the two curves rise together and are almost identical after epoch 250. On the multi ego-networks, GIN pulls ahead after about epoch 150, although the gap stays within one standard deviation. On ENZYMES, GraphSAGE never comes close to GIN at any epoch, which matches its low training accuracy. The selected epochs are single high points in noisy curves, which shows why figures chosen this way are somewhat optimistic.

Limitations and future work. Each configuration was run once, with one seed, so we cannot separate model differences from run-to-run variation beyond the fold spread. A nested selection rule, choosing the epoch on validation data, would remove the optimism noted above. We did not analyse which graphs each model gets wrong, because the driver saves accuracy per fold rather than per graph. Saving per-graph predictions would allow errors to be compared by size, density and class, and would test directly whether GraphSAGE's ENZYMES errors involve neighbourhoods with repeated labels. A WL kernel baseline would show how much of each result needs a neural model at all.

5.1 LLM as Judge

GIN's output is a class label scored against ground truth, so accuracy needs no judge. The part of the pipeline nobody has reviewed is the labels themselves, which come from TMDb and feed every graph in our data. We therefore used an LLM as an independent annotator. This differs slightly from the proposal, which suggested checking misclas-

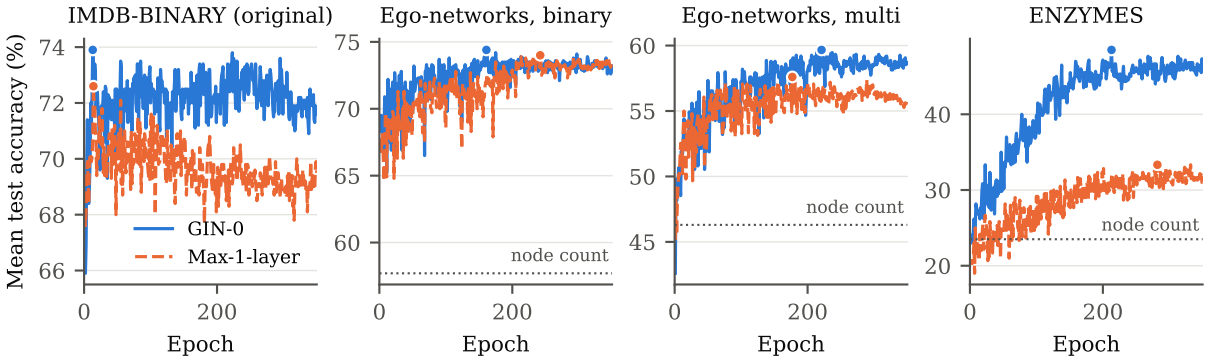


Figure 2: Test accuracy averaged over 10 folds at each epoch, GIN-0 (solid) and Max–1-layer (dashed). Dots mark the epoch chosen by the paper's selection rule; the dotted line is the node-count baseline.

```
You are annotating films by genre. For each film
below, choose the single genre that best describes
it from: Comedy, Romance, Science Fiction. Answer
UNSURE if you cannot tell from the title and cast.

Reply with only a JSON object mapping each film's
id to your answer, for example
{"0": "Comedy", "1": "UNSURE"}.

Films:
0: <title> | cast: <actor 1>, ..., <actor 8>
...
```

Figure 3: The judge's prompt for the multi task. The binary prompt lists Action and Romance instead. Line breaks are added here for width.

	Binary	Multi
Films judged	200	300
UNSURE	12	50
Agreement, all films	94.0%	80.3%
Agreement, decided films	100%	96.4%
Cohen's kappa, decided films	1.000	0.946

Table 4: LLM judge agreement with TMDb genre labels.

sified graphs: a random sample of films tests the labels themselves, independently of the model.

We sampled 100 films per genre at random from each dataset and showed Claude Sonnet 5.5 each film's title and top 8 billed actors in shuffled batches. The TMDb label was hidden and the judge had no tools. It chose one genre or answered UNSURE, and we measured agreement with TMDb using Cohen's kappa (Cohen, 1960), excluding UNSURE answers (Table 4). Figure 3 shows the prompt. The judge ran in an empty folder through the Claude Code command line, so it saw no project files, and answers were saved after each batch of 25 films. The script, `llm_judge.py`, and every answer are in our repository.

Agreement is very high, which supports using TMDb's labels. The judge decided 188 binary films and agreed on all of them, which is unsurprising given how distinct Action and Romance are. Most of the 9 disagreements on the multi task involve films that plausibly fit two genres, such as *Thor: Love and Thunder* (TMDb Comedy, judge Science Fiction) and *Good Luck to You, Leo Grande* (TMDb Romance, judge Comedy). The multi task is harder, with 50 UNSURE answers, 24 of them for Romance films, so that label appears the least obvious from title and cast. Agreement across all films is therefore lower there (80.3%). These figures need care. The judge chooses only among our target genres, so they do not show that a genre is a film's main one. It may also remember public genre listings, in which case it is not fully independent of TMDb, and the high figures cannot rule out shared errors. Kappa also excludes the films the judge found hardest.

The judge's most useful result was unplanned. When we first ran it on the earlier scrape, reading its disagreements exposed pre-2016 films in data meant to be 2016 onward, which led us to the date filter bug in Section 4. The figures above are from a rerun on the corrected films.

6 Ethical Considerations

Ethics statements are increasingly expected in published work. Benotti and Blackburn (2022) classify such sections by whether they cover research benefits, potential harms, and the groups affected, and Karamolegkou et al. (2025) find that the most common concerns in ACL ethics statements are bias, misuse and privacy, while many papers report no concerns at all. We follow that structure, and we also consider data licensing, which matters most here. The Australian Code (National Health and Medical Research Council and Australian Research

Council and Universities Australia, 2018) expects research to be honest, transparent and accountable, and to comply with relevant conditions.

Benefits. Replication checks whether a published claim holds, and the paper's public code and data made ours possible. Our negative result on the first scrape, and the checks it led to, may help others avoid the same mistake when building graph datasets.

Original work. IMDB-BINARY and IMDB-MULTI were released by Yanardag and Vishwanathan (2015), built from public IMDb film credits, and are distributed inside the official repository, whose code is under the MIT licence. Neither the paper nor the repository states terms for the data itself, so we used it only to replicate the published results. Our repository does not redistribute it: the setup script downloads it from the official repository. The graphs hold only anonymous node numbers and genre labels, with no names.

New data and terms of use. Our new data comes from the TMDb API, not from the IMDb website. TMDb's terms (TMDb, 2023) do not permit commercial use, and they treat training or validating machine learning systems on its content as commercial use needing a written agreement. They do not mention academic use, cache data for at most six months, and require attribution. Our proposal assumed academic use was unproblematic, and we did not read these clauses before collecting. **Licence and use declaration.** The new datasets and metadata in this project were created for non-commercial university coursework only. They must not be used for any commercial purpose. Any reuse, redistribution or publication beyond this assessment requires the appropriate licence and written permission from TMDb. No such licence is claimed here, and our repository does not grant one. Publishing a research paper on this data would need the same permission.

Attribution. This product uses the TMDb API but is not endorsed or certified by TMDb.

Privacy. Unlike the original graphs, our metadata holds real actors' names and TMDb identifiers, which we keep so that labels can be checked. These are public credits, and no consent was sought, which is normal for public filmographies. A genre-labelled collaboration graph still reveals who works with whom, so the graph files themselves use anonymous node numbers, and names appear only in the metadata.

Bias. Genre labels reflect the judgements of TMDb's contributors, and sampling by popularity favours mainstream, mostly English-language films. Results may not generalise to other film industries, and a model trained on this data could reinforce links between actors and genres.

Misuse. We see little direct risk in genre classification. The same methods could be used to profile people from their professional networks, which is one reason the declaration above limits reuse.

Honest reporting. We report the failed construction and the date filter bug, because omitting them would overstate the method's success on new data. We also state where our setup departs from the paper's, such as untuned hyperparameters and CPU runs.

7 Conclusions and Reflections

Replication was successful. The official code ran with one device fix, results fell within one standard deviation of the paper, and the Max–1-layer baseline matched closely. GIN also generalised to new data once the graphs carried real structure, beating the node-count baseline on every such dataset. Its theoretical advantage appeared clearly only on ENZYMES; on film ego-networks it was within noise, as on the original data. Higher expressive power therefore did not guarantee a measurable gain on every task.

The most useful lesson came from our own mistake. Mean density of exactly 1.000 was visible from the first dataset we built, and the accuracy drop looked like a finding about GIN until we tested the data itself. A simple baseline and a clique count would have caught it before any GPU time was spent, and both now run first. The same applies to the date filter, which failed silently, so a filter needs checking against its output. A further lesson is that a data source's terms need reading in full before collection. We also learned to measure before assuming: a short timing test showed the GPU was the slower choice for GraphSAGE on dense graphs.

Generative AI Acknowledgement

In line with Macquarie University's guidance on AI tools, I acknowledge:

- **Claude Code** (Anthropic, Claude Opus 5.5), August to November 2026:
  - helping me plan the project steps from the brief and rubric, and structure the proposal, presentation and this report;

  - writing, debugging and running the code, including the cross-validation driver, TMDb scraper, ego-network builder, ENZYMES converter, dataset checks, LLM judge and figure script, and running jobs on the university GPU server under my supervision;
  - diagnosing problems, including the GPU device bug, the slow Max–1-layer runs and the date filter bug;
  - drafting text, tables and the figure, suggesting sources, and writing the report in LaTeX.
- **Claude Sonnet 5.5** (Anthropic), used as the LLM judge in Section 5.1. Its outputs are evaluated as experimental results and were not used to write the report.

Verification.

- I obtained the TMDb API key, set up access to the university server, and ran the Google Colab notebooks.
- I checked results against the data, which led to corrections, for example the clique construction, the ignored date filter and the description of the paper's hyperparameters.
- I reviewed the sources and edited the text.

I remain responsible for the accuracy, references and content of this report. A dated project log is in our repository (`plan.md`).

References

Luciana Benotti and Patrick Blackburn. 2022. Ethics consideration sections in natural language processing papers. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 4509–4516.

Karsten M. Borgwardt, Cheng Soon Ong, Stefan Schönauer, S.V.N. Vishwanathan, Alex J. Smola, and Hans-Peter Kriegel. 2005. Protein function prediction via graph kernels. *Bioinformatics*, 21(suppl_1):i47–i56.

Jacob Cohen. 1960. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46.

William L. Hamilton, Rex Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NIPS)*, pages 1025–1035.

Antonia Karamolegkou, Sandrine Schiller Hansen, Ariadni Christopoulou, Filippos Stamatiou, Anne Lauscher, and Anders Søgaard. 2025. Ethical concern identification in NLP: A corpus of ACL anthology ethics statements. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics (NAACL)*. ArXiv:2411.07845.

Thomas N. Kipf and Max Welling. 2017. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*.

Christopher Morris, Nils M. Kriege, Franka Bause, Kristian Kersting, Petra Mutzel, and Marion Neumann. 2020. TUDataset: A collection of benchmark datasets for learning with graphs. In *ICML 2020 Workshop on Graph Representation Learning and Beyond*.

National Health and Medical Research Council and Australian Research Council and Universities Australia. 2018. Australian code for the responsible conduct of research. Commonwealth of Australia, Canberra.

TMDb. 2023. TMDB API terms of use. `https://www.themoviedb.org/api-terms-of-use`. Last updated 20 October 2023. Accessed 2026-11-01.

TMDb. 2026. The movie database (tmdb) api. `https://www.themoviedb.org/documentation/api`. Accessed 2026-08-29.

Boris Weisfeiler and A. A. Lehman. 1968. A reduction of a graph to a canonical form and an algebra arising during this reduction. *Nauchno-Technicheskaya Informatsia*, 2(9):12–16.

Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2019. How powerful are graph neural networks? In *International Conference on Learning Representations (ICLR)*.

Pinar Yanardag and S.V.N. Vishwanathan. 2015. Deep graph kernels. In *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 1365–1374.